# Troubleshooting: boundary models, 1

Daniel C. Reuman[1] and Emilio Berti[2]

[1]Department of Ecology and Evolutionary Biology and Center for Ecological Research, University of Kansas
[2]German Center for Integrative Biodiversity Science

**Abstract**

We here show a troublshooting example in the case where the best model considered does not show adequate evidence that the likelihood function was fully optimized, the best parameters obtained by optimization show $p_d$ close to 1, and the profile for $p_d$ is not dome shaped. This is a common case. The boundary model with $p_d = 1$ is used. This example is based on occurrence data from GBIF for *Blarina carolinensis*, the southern short-tailed shrew.

The southern short-tailed shrew, *Blarina carolinensis*, is found in the southeastern United States.

We start by loading in the data:

```
env_array = readRDS("blarina_carolinensis_env.Rds")
dim(env_array)

## [1] 1156   39    6

dimnames(env_array)[[3]]

## [1] "BIO01" "BIO10" "BIO11" "BIO12" "BIO16" "BIO17"

occ = readRDS("blarina_carolinensis_occ.Rds")
length(occ)

## [1] 1156
```

Here, there are 6 environmental variables recorded for 39 years in 1156 locations, with accompanying detections and pseudo-absences in the variable `occ`. The first three environmental variables (BIO1, BIO10, BIO11) are temperature variables, and the last three (BIO12, BIO16, BIO17) are precipitation variables.

Now look at the distributions of values of environmental variables to make sure they are not wildly divergent, which would cause problems for optimization:

```
apply(FUN=quantile, X=env_array, MARGIN=3,prob=c(.025,.25,.5,.75,.975))

##          BIO01    BIO10     BIO11     BIO12    BIO16     BIO17
## 2.5%  11.61870 21.70901  1.477658  76.36044  9.12790  2.133369
## 25%   15.31648 25.24791  6.289062 108.62843 13.18716  4.505094
```

```
## 50%   17.16010 26.45451  9.055910 126.29746 15.76004  5.947245
## 75%   19.20414 27.43751 11.991494 146.79118 18.83666  7.377723
## 97.5% 22.55861 29.21995 17.613174 190.28207 27.04961 10.422263
```

Looks basically OK.

Now fit 15 models, each from 25 starting conditions:

```
models = matrix(c(1,0,0,0,0,0,
                  0,1,0,0,0,0,
                  0,0,1,0,0,0,
                  0,0,0,1,0,0,
                  0,0,0,0,1,0,
                  0,0,0,0,0,1,
                  1,0,0,1,0,0,
                  1,0,0,0,1,0,
                  1,0,0,0,0,1,
                  0,1,0,1,0,0,
                  0,1,0,0,1,0,
                  0,1,0,0,0,1,
                  0,0,1,1,0,0,
                  0,0,1,0,1,0,
                  0,0,1,0,0,1), nrow=15, byrow=TRUE)
all_model_results = list()
for (i in 1:nrow(models))
{
  env_dat = env_array[,,models[i,]==1,drop=FALSE]
  starts = xsdmMle::start_parms(env_dat,num_starts=25)
  all_optim_results = list()
  for (j in 1:nrow(starts))
  {
    all_optim_results[[j]] = optim(par=starts[j,],fn=xsdmMle::loglik_math,
                                   method="BFGS",
                                   env_dat=env_dat, occ=occ,negative=TRUE,
                                   control=list(trace=0))
  }
  all_model_results[[i]] = all_optim_results
}
```

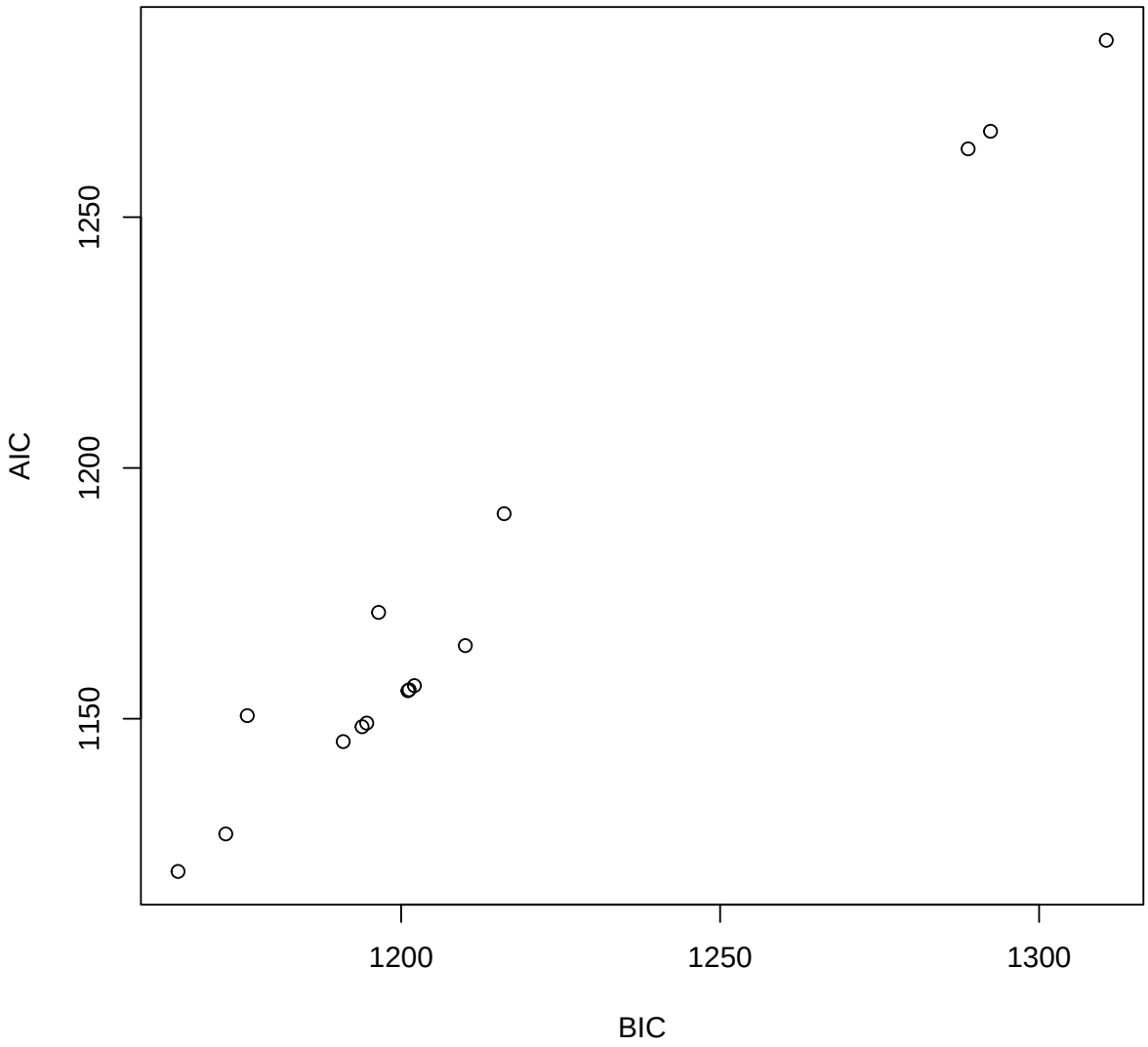
Rank the models by BIC, bearing in mind that we've been working with the negative of the likelihood:

```
model_BICs = sapply(X=all_model_results,
                    FUN=function(x){
                      best_loglik = min(sapply(X=x, FUN=function(y){y$value}))
                      num_parms = length(x[[1]]$par)
                      n = length(occ)
                      BIC = 2*best_loglik + num_parms*log(n)
```

```
                        return(BIC)
                      }
                    )
```

Also by AIC:

```
model_AICs = sapply(X=all_model_results,
                    FUN=function(x){
                      best_loglik = min(sapply(X=x, FUN=function(y){y$value}))
                      num_parms = length(x[[1]]$par)
                      AIC = 2*best_loglik + 2*num_parms
                      return(AIC)
                    }
                   )
inds = order(model_BICs)
rbind(model_BICs[inds],model_AICs[inds])

##          [,1]     [,2]     [,3]     [,4]     [,5]     [,6]     [,7]     [,8]
## [1,] 1165.031 1172.509 1175.884 1190.920 1193.879 1194.610 1196.464 1201.064
## [2,] 1119.557 1127.035 1150.620 1145.446 1148.404 1149.136 1171.200 1155.590
##          [,9]    [,10]    [,11]    [,12]    [,13]    [,14]    [,15]
## [1,] 1201.253 1202.077 1210.059 1216.145 1288.876 1292.384 1310.530
## [2,] 1155.779 1156.602 1164.584 1190.882 1263.613 1267.121 1285.266

plot(model_BICs,model_AICs,type="p",xlab="BIC",ylab="AIC")
```




```
order(model_BICs)
```

```
##  [1]  9 12  2  7  8 15  1 11 10 13 14  3  5  4  6
```

```
order(model_AICs)
```

```
##  [1]  9 12  7  8 15  2 11 10 13 14  1  3  5  4  6
```

Looks like in this case the AIC and the BIC are pretty aligned with each other. Look at the best two model models:

```
models[order(model_BICs)[1:2],]
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    1    0    0    0    0    1
## [2,]    0    1    0    0    0    1
```

So these are two-variable models.

Let's use the best model. Optimize it a bit harder:

```
i=9
env_dat = env_array[,,models[i,]==1,drop=FALSE]
starts = xsdmMle::start_parms(env_dat,num_starts=100)
best_model_results = list()
for (j in 1:nrow(starts))
{
  best_model_results[[j]] = optim(par=starts[j,],fn=xsdmMle::loglik_math,
                                  method="BFGS",
                                  env_dat=env_dat, occ=occ,negative=TRUE,
                                  control=list(trace=0))
}
values = sapply(X=best_model_results, FUN=function(y){y$value})
inds = order(values)
best_model_results = best_model_results[inds]
min(sapply(X=all_model_results[[i]], FUN=function(y){y$value}))
```

```
## [1] 550.7784
```

```
best_model_results[[1]]$value
```

```
## [1] 550.7784
```

Pretty similar, so we HAD pretty much fully optimized before.

Now have a look at the result for this model.

```
examine_optim_results = function(optim_results,mask=NULL)
{
  #put optimization results in order from best to worst
  bestlogliks = sapply(X=optim_results,FUN=function(x){x$value})
  inds = order(bestlogliks)
  bestlogliks = bestlogliks[inds]
  optim_results = optim_results[inds]

  #model convergence
  convergences = sapply(X=optim_results,FUN=function(x){x$convergence})

  #compute distances to the best result in parameter space
  best_parms_math = optim_results[[1]]$par
  parms_dists_to_best = lapply(
```

```
    X=optim_results,
    FUN=function(x){
      xsdmMle::distance_between_params_hungarian(
        x$par,
        best_parms_math,
        mask=mask,
        GiveClosestRep=TRUE)
    }
  )
  parms_dists = sapply(X=parms_dists_to_best, FUN=function(x){x$distance})

  #look at the best 5 results in parameter space
  bestparms=sapply(X=parms_dists_to_best, FUN=function(x){unlist(x$representative)})

  #put it all together
  return(rbind(bestlogliks,convergences,parms_dists,bestparms))
}

h=examine_optim_results(best_model_results)
t(h[,1:8])
```

```
##      bestlogliks convergences  parms_dists      mu1      mu2  sigltil1 sigltil2
## [1,]    550.7784            0 0.000000e+00 14.26892 3.549589 0.1481589 4.892067
## [2,]    550.7784            0 1.160121e-04 14.26893 3.549568 0.1481564 4.891978
## [3,]    550.7784            0 6.023232e-05 14.26890 3.549594 0.1481577 4.892054
## [4,]    550.7784            0 1.875635e-04 14.26883 3.549644 0.1481589 4.892080
## [5,]    550.7784            0 1.668666e-04 14.26890 3.549625 0.1481618 4.892080
## [6,]    550.7784            0 9.599053e-05 14.26892 3.549582 0.1481568 4.892010
## [7,]    550.7784            0 7.617513e-05 14.26891 3.549591 0.1481573 4.892030
## [8,]    550.7784            0 2.926731e-04 14.26886 3.549711 0.1481638 4.891906
##      sigrtil1  sigrtil2       ctil        pd    o_mat1    o_mat2     o_mat3
## [1,] 4.602571 0.4576403 -0.1921821 0.9999959 0.2429329 0.9700431 -0.9700431
## [2,] 4.602647 0.4576429 -0.1921954 0.9999953 0.2429328 0.9700431 -0.9700431
## [3,] 4.602665 0.4576380 -0.1921622 0.9999939 0.2429355 0.9700424 -0.9700424
## [4,] 4.602568 0.4576090 -0.1922089 0.9999936 0.2429384 0.9700417 -0.9700417
## [5,] 4.602517 0.4576222 -0.1921961 0.9999902 0.2429344 0.9700427 -0.9700427
## [6,] 4.602692 0.4576394 -0.1921811 0.9999901 0.2429346 0.9700427 -0.9700427
## [7,] 4.602668 0.4576376 -0.1921769 0.9999886 0.2429354 0.9700425 -0.9700425
## [8,] 4.602530 0.4576222 -0.1922789 0.9999887 0.2429438 0.9700404 -0.9700404
##         o_mat4
## [1,] 0.2429329
## [2,] 0.2429328
## [3,] 0.2429355
## [4,] 0.2429384
## [5,] 0.2429344
## [6,] 0.2429346
## [7,] 0.2429354
## [8,] 0.2429438
```

This looks good, in the sense that it looks like multiple optimizations arrived at about the same place in parameter space, which is some evidence that we may have found the global maximum of the likelihood function The only problem is that $p_d$ is very close to 1.

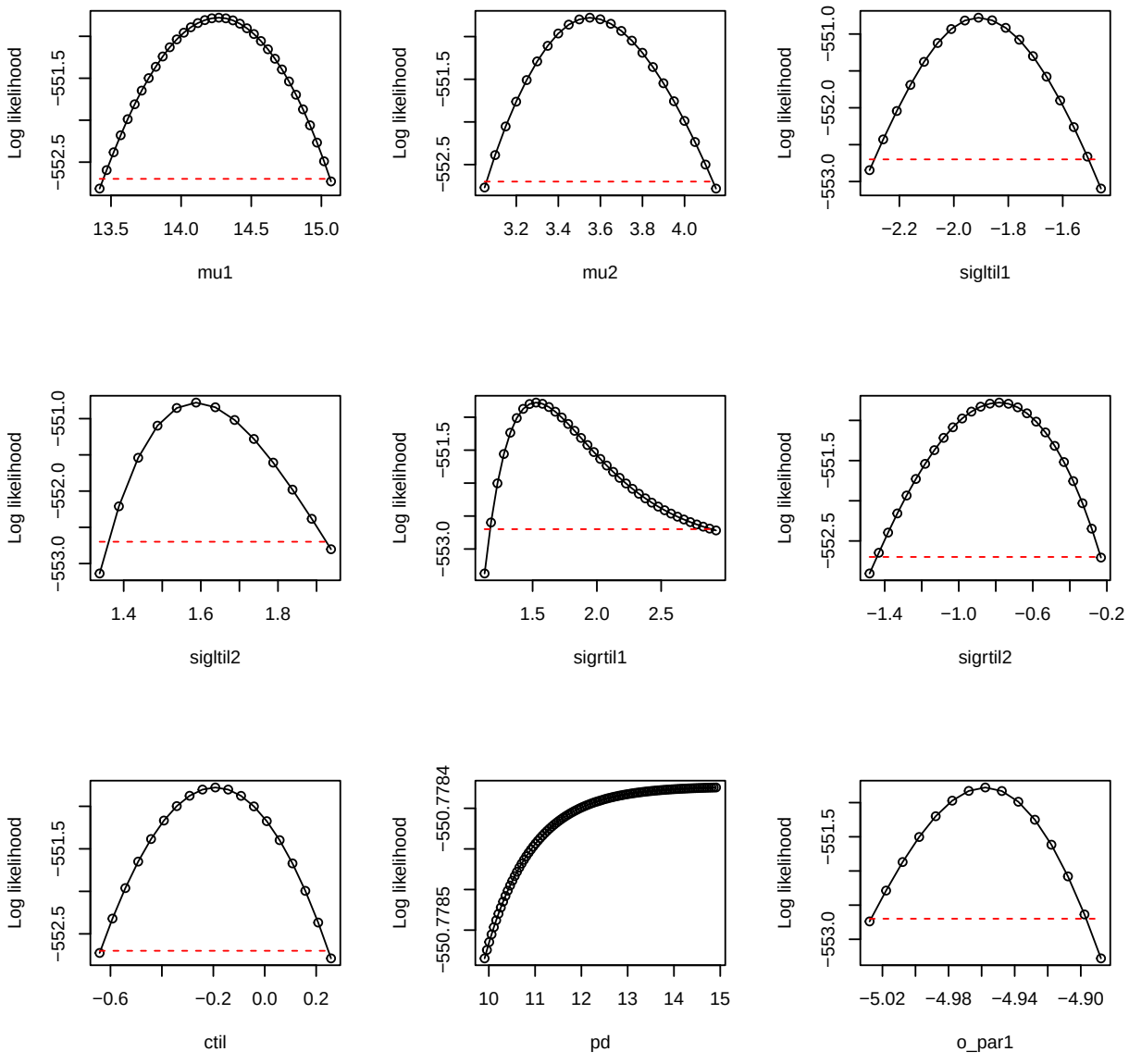
Let's profile this model and see what we get:

```
pnames=names(xsdmMle::make_mask_names(2))

all_profiles=list()
linc = c(rep(0.05,8),0.01)
rinc = c(rep(0.05,8),0.01)
for (counter in 1:9)
{
  all_profiles[[counter]] = xsdmMle::profile_likelihood(
                              profile_parameter=pnames[counter],
                              increment_left=linc[counter],
                              increment_right=rinc[counter],
                              num_steps_left=50,
                              num_steps_right=50,
                              alpha=0.95,
                              optim_param_vector=best_model_results[[1]]$par,
                              env_dat=env_dat,
                              occ=occ,
                              mask=NULL,
                              num_threads=6
                            )
}
names(all_profiles) = pnames
```

Now plot these profiles:

```
plot_tool=function(ap,index)
{
  x=ap[[index]]$profile$value_math
  y=ap[[index]]$profile$loglik
  xlab=names(ap)[index]
  thresh=ap[[index]]$threshold
  plot(x,y,
       type="o",xlab=xlab,
       ylab="Log likelihood")
  lines(range(x),rep(thresh,2),type="l",
        lty="dashed",col="red")
}

par(mfrow=c(3,3))
plot_tool(all_profiles,1)
plot_tool(all_profiles,2)
```

```
plot_tool(all_profiles,3)
plot_tool(all_profiles,4)
plot_tool(all_profiles,5)
plot_tool(all_profiles,6)
plot_tool(all_profiles,7)
plot_tool(all_profiles,8)
plot_tool(all_profiles,9)
```




So yes, there is the expected problem with pd. This is a common problem and it manifests in the manner illustrated here.

The solution is to use the boundary model with $p_d = 1$:

```
env_dat = env_array[,,models[i,]==1,drop=FALSE]
dim(env_dat)

## [1] 1156   39    2

mask=c(pd=Inf) #use Inf because masks are given on the math scale
mask

##  pd
## Inf

new_starts = xsdmMle::start_parms(env_dat[occ==1,,,drop=FALSE],
                                  mask=mask,num_starts=100)
head(new_starts)

## # A tibble: 6 x 8
##     mu1   mu2 sigltil1 sigltil2 sigrtil1 sigrtil2   ctil o_par1
##   <dbl> <dbl>    <dbl>    <dbl>    <dbl>    <dbl>  <dbl>  <dbl>
## 1  15.6  5.66   0.232    1.09      0.562    1.27  -1.19    3.68
## 2  18.1  8.30   0.925    0.397     1.26     0.572 -0.618  -5.74
## 3  19.3  4.34   0.579    0.744     1.60     0.919 -0.905   8.39
## 4  16.9  6.98  -0.115    0.0504    0.908    0.226 -1.48   -1.03
## 5  17.5  3.68   0.752    1.26      1.43     0.399 -1.34   -8.10
## 6  19.9  6.32   0.0587   0.570     0.735    1.09  -0.762   1.33

bdry_optim_results = list()
for (j in 1:nrow(new_starts))
{
  bdry_optim_results[[j]] = optim(par=new_starts[j,],fn=xsdmMle::loglik_math,
                                  method="BFGS",
                                  env_dat=env_dat,occ=occ,mask=mask,negative=TRUE,
                                  control=list(trace=0,maxit=500))
}
```

Have a look at these results:

```
h=examine_optim_results(bdry_optim_results,mask=mask)
t(h[,1:10])

##       bestlogliks convergences  parms_dists      mu1      mu2 sigltil1
##  [1,]    550.7784            0 0.000000e+00 14.26900 3.549559 4.602727
##  [2,]    550.7784            0 7.005986e-05 14.26898 3.549575 4.602621
##  [3,]    550.7784            0 2.575040e-04 14.26893 3.549564 4.602640
##  [4,]    550.7784            0 6.015509e-05 14.26900 3.549560 4.602613
##  [5,]    550.7784            0 1.353738e-04 14.26894 3.549589 4.602636
##  [6,]    550.7784            0 1.720022e-04 14.26892 3.549588 4.602699
##  [7,]    550.7784            0 1.700789e-04 14.26892 3.549588 4.602682
##  [8,]    550.7784            0 2.189342e-04 14.26900 3.549554 4.602524
```

```
##  [9,]    550.7784            0 3.631189e-04 14.26890 3.549573 4.602579
## [10,]    550.7784            0 1.930579e-04 14.26892 3.549588 4.602666
##        sigltil2  sigrtil1 sigrtil2       ctil pd     o_mat1     o_mat2
##  [1,] 0.4576672 0.1481599 4.892012 -0.1921522  1 -0.2429288 -0.9700441
##  [2,] 0.4576555 0.1481604 4.892036 -0.1921766  1 -0.2429312 -0.9700435
##  [3,] 0.4576443 0.1481550 4.892054 -0.1921698  1 -0.2429308 -0.9700436
##  [4,] 0.4576663 0.1481612 4.892099 -0.1921537  1 -0.2429290 -0.9700441
##  [5,] 0.4576433 0.1481595 4.892003 -0.1921777  1 -0.2429296 -0.9700439
##  [6,] 0.4576423 0.1481578 4.892052 -0.1921682  1 -0.2429326 -0.9700432
##  [7,] 0.4576431 0.1481580 4.892035 -0.1921824  1 -0.2429318 -0.9700434
##  [8,] 0.4576718 0.1481553 4.891997 -0.1922022  1 -0.2429307 -0.9700436
##  [9,] 0.4576306 0.1481534 4.892075 -0.1921963  1 -0.2429314 -0.9700435
## [10,] 0.4576403 0.1481574 4.892029 -0.1921747  1 -0.2429334 -0.9700430
##          o_mat3     o_mat4
##  [1,] 0.9700441 -0.2429288
##  [2,] 0.9700435 -0.2429312
##  [3,] 0.9700436 -0.2429308
##  [4,] 0.9700441 -0.2429290
##  [5,] 0.9700439 -0.2429296
##  [6,] 0.9700432 -0.2429326
##  [7,] 0.9700434 -0.2429318
##  [8,] 0.9700436 -0.2429307
##  [9,] 0.9700435 -0.2429314
## [10,] 0.9700430 -0.2429334
```

This looks like enough of them converged to the same thing to say that it looks like I successfully optimized. Get the likelihood:

```
values = sapply(X=bdry_optim_results, FUN=function(y){y$value})
inds = order(values)
bdry_optim_results = bdry_optim_results[inds]
best_model_results[[1]]$value

## [1] 550.7784

bdry_optim_results[[1]]$value

## [1] 550.7784
```

So the likelihood is the same as for the previous (non-boundary) model. But we have one less parameter that has been fitted. Get the AIC and BIC to see the effects:

```
AIC = unname(2*bdry_optim_results[[1]]$value+
               2*length(bdry_optim_results[[1]]$par))
AIC_old = unname(2*best_model_results[[1]]$value+
                   2*length(best_model_results[[1]]$par))
AIC

## [1] 1117.557
```

```
AIC_old

## [1] 1119.557

AIC-AIC_old

## [1] -2.000035

BIC = unname(2*bdry_optim_results[[1]]$value+
               log(length(occ))*length(bdry_optim_results[[1]]$par))
BIC_old = unname(2*best_model_results[[1]]$value+
               log(length(occ))*length(best_model_results[[1]]$par))
BIC

## [1] 1157.979

BIC_old

## [1] 1165.031

BIC-BIC_old

## [1] -7.052756

log(length(occ))

## [1] 7.052721
```

So the AIC and BIC are better for the boundary model compared to the earlier model, and by the expected amounts. We go with the simpler model.

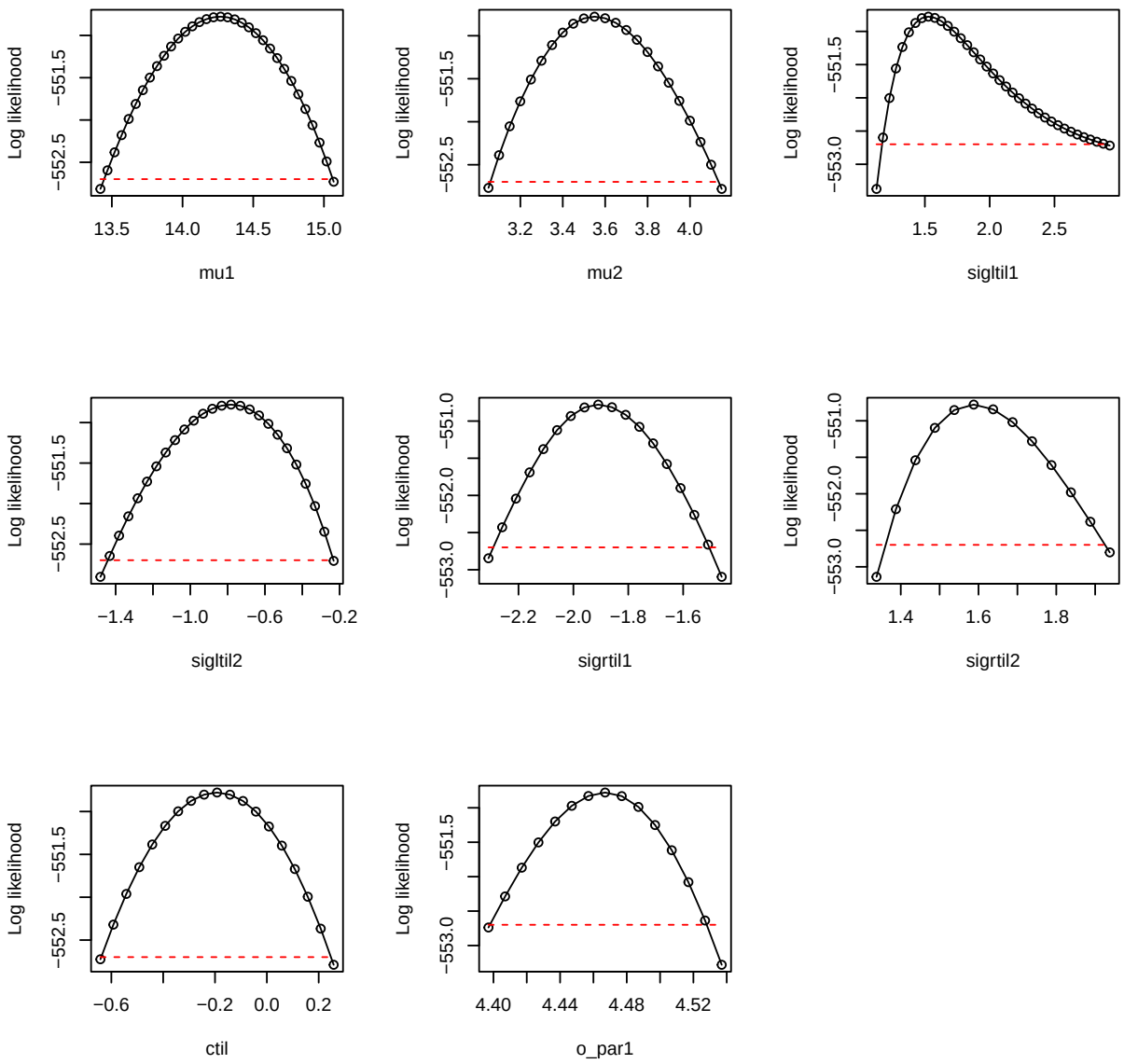
Let's profile this boundary model and see what we get:

```
pnames=names(xsdmMle::make_mask_names(2))
pnames=pnames[pnames!="pd"]
mask

##  pd
## Inf

all_bdry_profiles=list()
linc = c(rep(0.05,7),0.01)
rinc = c(rep(0.05,7),0.01)
for (counter in 1:8)
{
  all_bdry_profiles[[counter]] = xsdmMle::profile_likelihood(
                               profile_parameter=pnames[counter],
                               increment_left=linc[counter],
                               increment_right=rinc[counter],
                               num_steps_left=50,
```

```
                                num_steps_right=50,
                                alpha=0.95,
                                optim_param_vector=bdry_optim_results[[1]]$par,
                                env_dat=env_dat,
                                occ=occ,
                                mask=mask,
                                num_threads=6
                              )
}
names(all_bdry_profiles) = pnames
```

Now plot these profiles:

```
par(mfrow=c(3,3))
plot_tool(all_bdry_profiles,1)
plot_tool(all_bdry_profiles,2)
plot_tool(all_bdry_profiles,3)
plot_tool(all_bdry_profiles,4)
plot_tool(all_bdry_profiles,5)
plot_tool(all_bdry_profiles,6)
plot_tool(all_bdry_profiles,7)
plot_tool(all_bdry_profiles,8)
```




These are dome-shaped, suggesting the likelihood function is well-behaved in the vacinity of the maximum we found, and inferences can be made. Note these profiles look essentially the same as those obtained previously, except $\vec{\sigma}_L$ and $\vec{\sigma}_R$ have been switched (which can happen, given redundancy in parameters explained in "How to fit xsdm models with species occurrence data using xsdmMle"). The overall lesson is, when optimizing the xsdm likelihood function gives a best model with $p_d$ close to 1 and a non-dome-shaped profile for $p_d$, use the boundary model with $p_d = 1$.